



Streamlining Data Pipelines Creation at Stripe with Airflow

Jiayu Yi

Senior Software Engineer, Stripe



Software Engineer on the Data Transformation Team

Been at Stripe for 4 years

Previously on the Data Orchestration team for 3.5 years

What is Stripe?

Global GDP running on Stripe: 1.67704930%

Financial infrastructure to grow your revenue.
Accept payments, offer financial services, and
implement custom revenue models—from
your first transaction to your billionth.

Get started >

 Sign up with Google

What is Stripe?

The backbone of global commerce

135+

currencies and payment
methods supported

\$1.9T

in payments volume
processed in 2025

99.999%

historical uptime
for Stripe services

200M+

active subscriptions
managed on Stripe Billing



Why is creating data pipelines difficult at Stripe?

01

Data Discovery

02

**Airflow
Authoring**

03

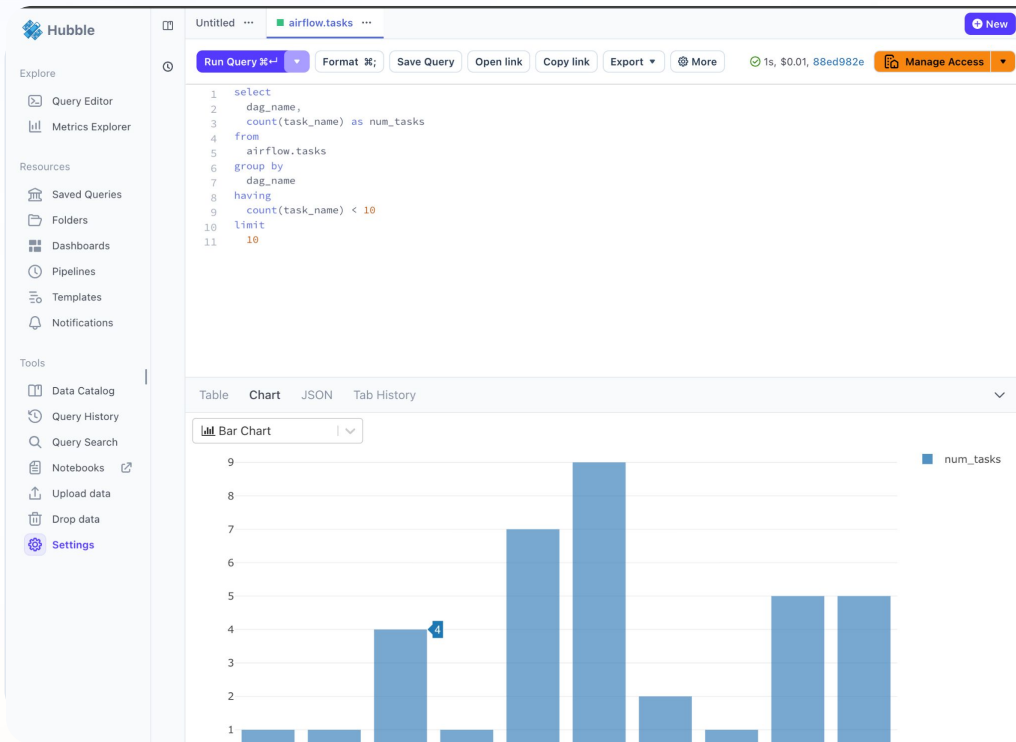
**Governance
Overhead**

Data Discovery



In-house Analytics Platform (Hubble!)

Hubble acts as the main entry point for users to explore data and create rich, interactive reports.



Data Discovery

Different Data Formats

Inputs exist in many formats — Iceberg, raw Parquet, views and more.



In-house Analytics Platform (Hubble!)

Hubble acts as the main entry point for users to explore data and create rich, interactive reports.



Storage Abstraction Layer

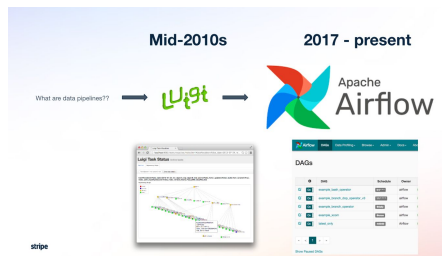
A unified way to represent different input types in code to simplify pipeline configurations.



But these solutions are still not enough.

Airflow authoring *at Stripe* is complex

If you attended our presentation last year at the Airflow Summit...



```
daily_revenue.py

from tasks import task as t

class DailyRevenueSummary(t.SparkTask):

    def requires(self):
        # Upstream deps
        return [
            PaymentsTransactions(),
            MerchantAccounts(),
        ]

    def output(self):
        # Dataset produced
        return t.Table(
            "analytics.daily_revenue"
        )
```

1. Circular Dependency Risks

Tasks must be carefully written to avoid dependency cycles.

2. Complex Dataset Tracing

Users must manually trace upstream task producers in code.

3. Two Airflow Environments

Ongoing migrations from Airflow 1 to Airflow 2 create overhead.

4. Multiple Execution Engines

Different engines and output types require custom implementations.

Governance overhead



Alerting

Ensuring prompt notifications when pipelines fail or data delivery is delayed.



Ownership

Assigning clear responsibility and main contact points for every data asset.



Scheduling

Managing complex execution sequences, triggers, and timing dependencies.



Freshness SLA

Defining and enforcing strict time-based data delivery guarantees.

Streamline Pipeline Creation with YAML

```
daily_revenue_summary.yaml

- name: analytics.daily_revenue_summary
  description: "Daily revenue aggregated by merchant"
  locations:
  iceberg:
  retention: 12 months
  source:
  query:
  dialect: SPARK_SQL
  file: models/sql/main/daily_revenue_summary.sql
  time_window: TIME_WINDOW_DAILY
  inputs:
  - payments.transactions
  - merchants.accounts
  incremental: true
  schema:
  columns:
  - name: merchant_id, data_type: string
  - name: revenue, data_type: double
  extensions:
  airflow:
  start_date: '2025-01-01T00:00:00Z'
```

Industry Validation

This approach turns out not to be unique to Stripe.

"Streamlining DAG Creation with YAML in Large Organizations"

Howie Wang, OpenAI

[Airflow Summit 2024](#)

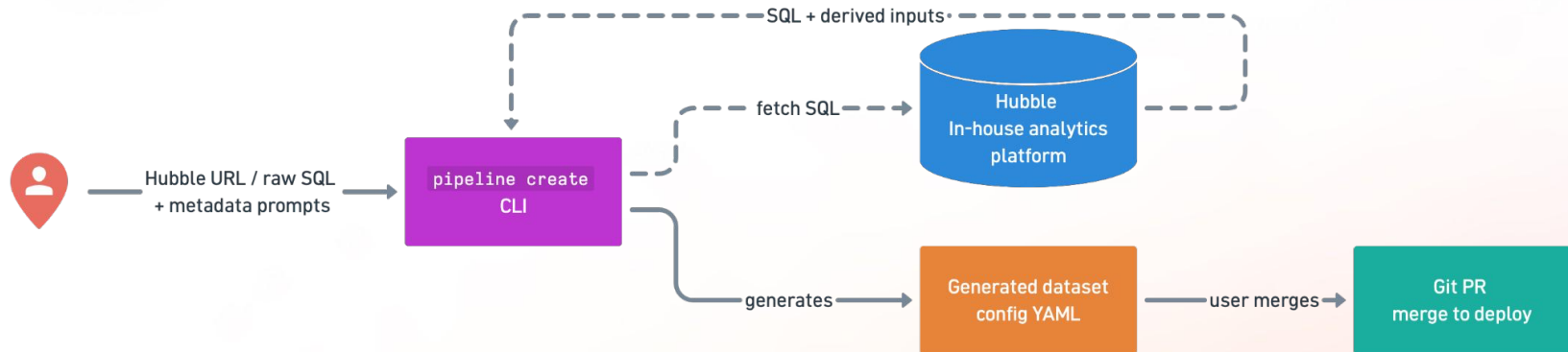
Streamline Pipeline Creation with YAML



Distinction

The YAML file is not meant to be hand-authored.

Instead, we built a CLI suite that generates and maintains these files for users.

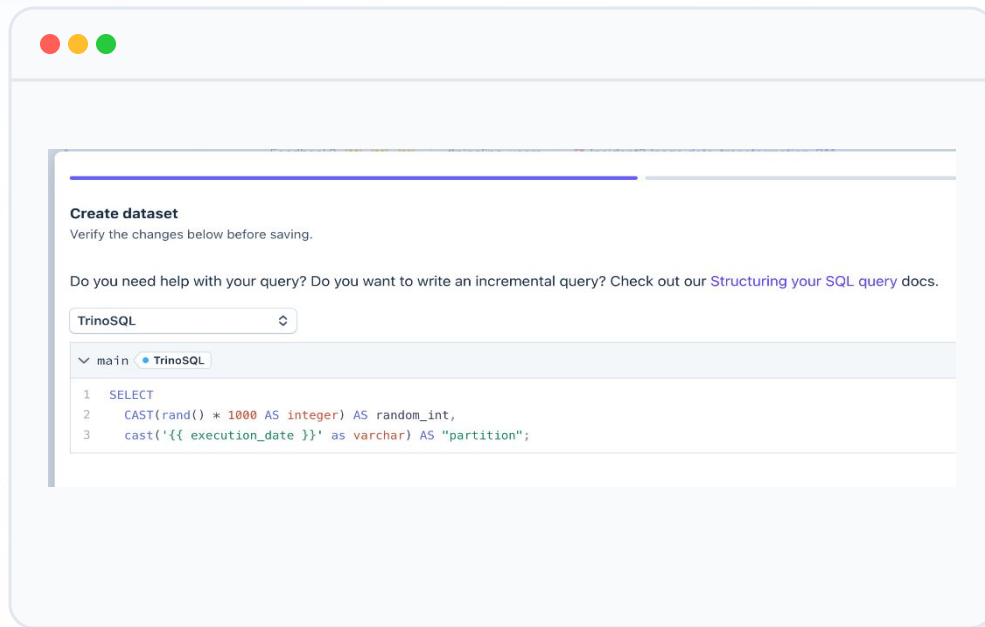


Streamline Pipeline Creation with Web UI



Create dataset

Verify and iterate on your queries directly in the Web UI before saving. The platform provides real-time syntax validation.

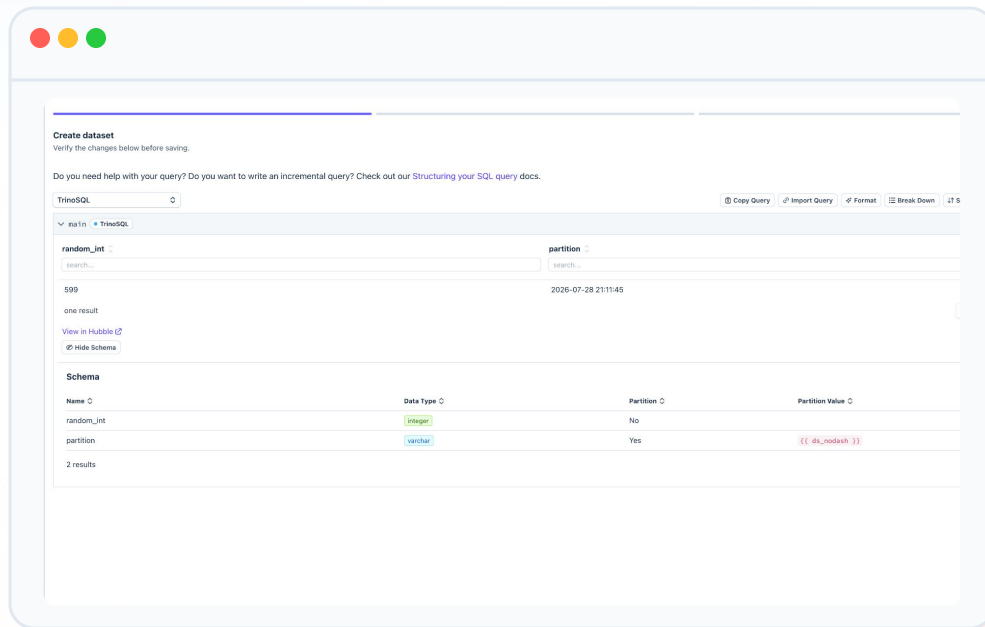


Streamline Pipeline Creation with Web UI



Preview results

Preview the expected query results directly within the in-house analytics platform before committing the pipeline change.



Streamline Pipeline Creation with Web UI



Fill in metadata

Easily provide essential metadata for your dataset directly in the Web UI, ensuring proper documentation, ownership, and scheduling from the start.

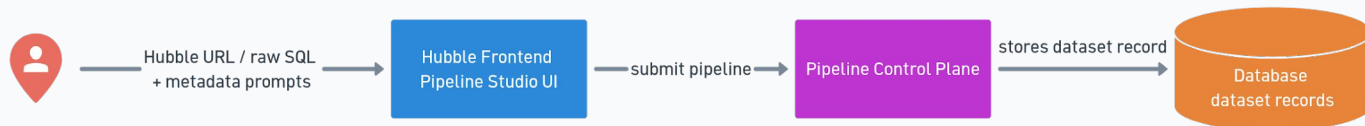
The screenshot shows a web browser window with a 'Create dataset' form. The form is titled 'Create dataset' and includes a sub-header 'Verify the changes below before saving.' The form contains several sections with labels and input fields:

- Name (required)**: A text input field with the placeholder 'namespace.table_name'.
- Project (required)**: A text input field with the placeholder 'helenyi-test-pipeline'.
- Description (required)**: A text area with the placeholder 'Write a brief description of what the dataset is used for. Supports markdown. What is this dataset used for?'.
- Frequency (required)**: A dropdown menu with 'Daily' selected.
- Incremental table**: A checkbox labeled 'Incremental table' which is currently unchecked. Below it, a note states: 'If incremental, the query should only return new data to append.'
- Reader LDAP Groups (required)**: A text input field with the placeholder 'Add the LDAP groups that can read this dataset'.
- Upstreams**: A text input field with the placeholder 'If a dataset is logged off, your task will not wait for it to finish before running. Your query has no input tables'.
- Start Date**: A date and time picker showing '2026 / 07 / 29 1:00:00 Z'.
- Schema**: A text input field with the placeholder 'The data types of the columns in your query. Click on a column to sort.' and a checkbox labeled 'Inferred' which is currently unchecked.

Streamline Pipeline Creation with Web UI

DISTINCTION

More simplified UX workflow for users

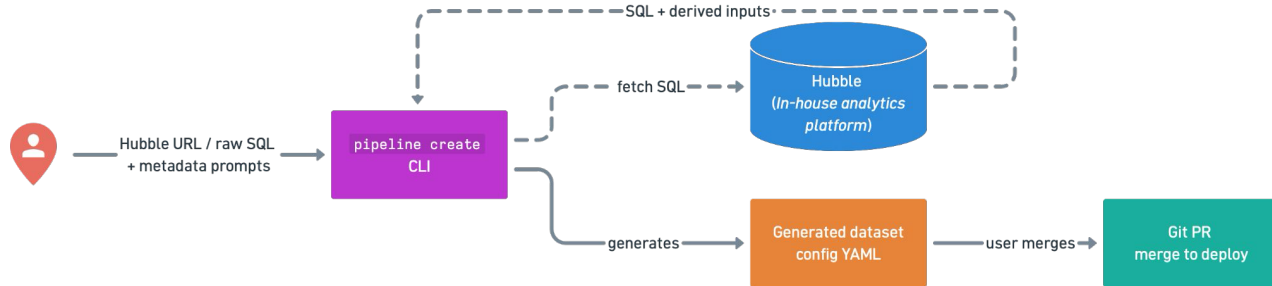


Bring it all together

Developer Flow

Git based path (aka Simplified Pipelines)

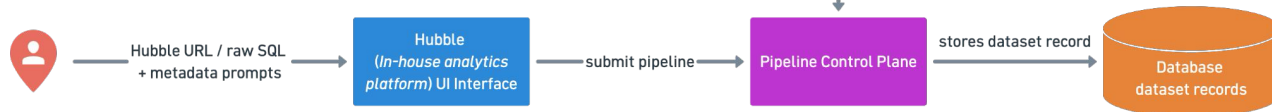
Create and manage configurations via CLI and version control.



Interactive Flow

Web based path (aka Pipeline Studio)

Build, configure, and submit pipelines directly within the Web UI.



Compare the Solutions

3X

Adoption Rate

The web-based solution has three times the adoption rate compared to the git-based solution.



Shift in User Needs

Users increasingly prioritize governance and tracking features:

- An audit trail of their dataset history
- Richer review and collaboration tooling



Integrated Flow

The UI seamlessly guides users to create PRs, effectively bridging the simplicity of web authoring with the robust change control of Git.

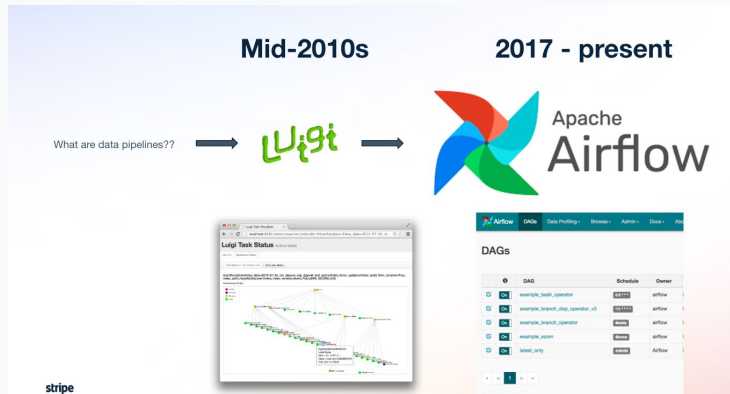
Create PR through Web UI



Stripe's take on Dataset Centric Scheduling

Luigi-Inherited DSL Complexity

Because of the custom DSL Stripe inherited from Luigi, adopting Airflow's native dataset scheduling wasn't straightforward.



```
daily_revenue.py

from tasks import task as t

class DailyRevenueSummary(t.SparkTask):

    def requires(self):
        # Upstream deps instantiated directly
        return [
            PaymentsTransactions(),
            MerchantAccounts(),
        ]

    def output(self):
        # Dataset this task produces.
        # CI reads this to build the
        # dataset-to-task mapping.
        return
        t.Table("analytics.daily_revenue_summary")
```

Stripe's take on Dataset Centric Scheduling

Dataset-to-Task Mapping

payments.transactions

→ PaymentsTransactionsTask

merchants.accounts

→ MerchantAccountsTask

```
dataset_mapping.json

{
  "payments.transactions": {
    "dag_id": "payments_pipeline",
    "task_id": "PaymentsTransactionsTask"
  },
  "merchants.accounts": {
    "dag_id": "merchants_pipeline",
    "task_id": "MerchantAccountsTask"
  }
}
```

Stripe's take on Dataset Centric Scheduling



Hermetic Datasets

Datasets produced by tasks that do not change between deployments.

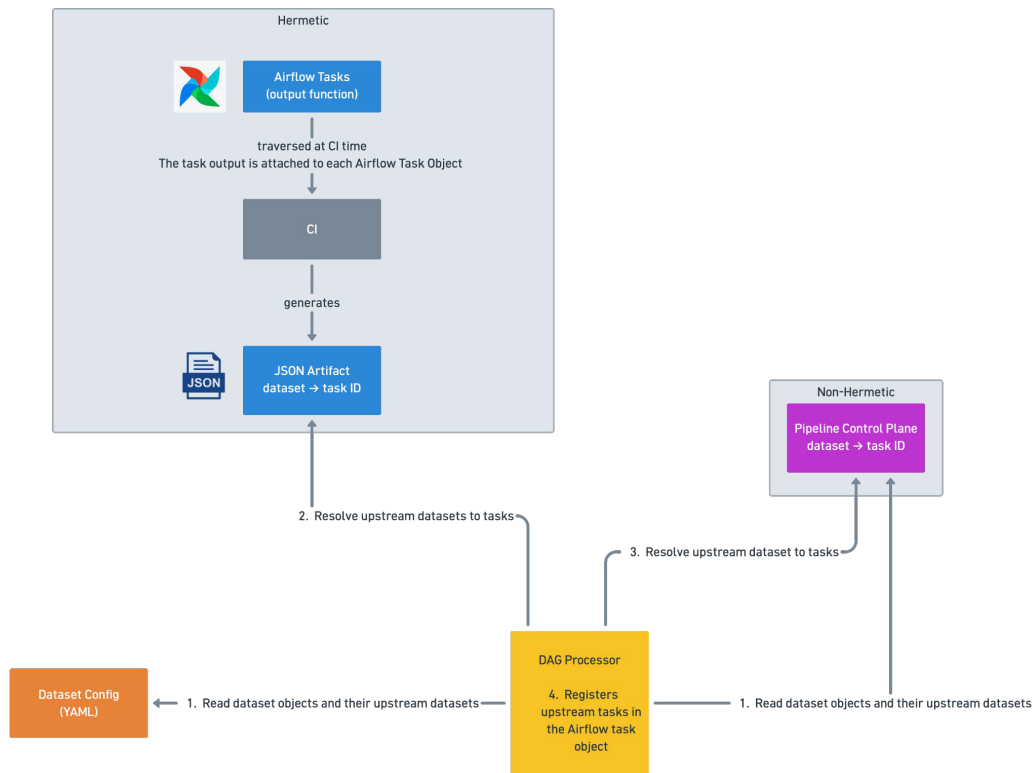
Almost all Non Pipeline Studio datasets



Non-Hermetic Datasets

Produced solely by Pipeline Studio dynamically.

Stripe's take on Dataset Centric Scheduling



Stripe's take on Dataset Centric Scheduling

Dataset YAML file

```
daily_revenue_summary.yaml

- name: analytics.daily_revenue_summary
description: "Daily revenue aggregated by merchant"
locations:
iceberg:
retention: 12 months
dialect: SPARK_SQL
file: models/sql/main/daily_revenue_summary.sql
time_window: TIME_WINDOW_DAILY
inputs:
- payments.transactions
- merchants.accounts
incremental: true
columns:
- name: merchant_id, data_type: string
- name: revenue, data_type: double
airflow:
start_date: '2025-01-01T00:00:00Z'
```

The Task Code You'd Have to Write by Hand

```
daily_revenue.py

from tasks import task as t
class DailyRevenueSummary(t.SparkTask):
    def requires(self):
        # Upstream deps instantiated directly
        return [
            ExternalTaskSensor(task_id="payments.transactions.task"
            ,
            ExternalTaskSensor(task_id="merchants.accounts.task"),
        ]
```

The dataset-to-task mapping declares that there's a mapping between input datasets and task: **payments.transactions** maps to **payments.transactions.task** and **merchants.accounts** maps to **merchants.accounts.task**. We use sensor tasks to enforce this dependency

More Notes on Using External Task Sensors

Cross-DAG Dependencies

Internally at Stripe, we use external task sensors to establish dependencies between tasks originating from different DAGs.

Adoption of Airflow Triggerer

This setup inevitably leads to a massive number of sensors. To handle this efficiently, we utilize the Airflow Triggerer to ensure active worker slots are not consumed while waiting for upstream dependencies.

Key Takeaways

01

Config over Code

Config over code makes pipeline creation ergonomic at scale.

02

Decoupled Logic

Separate your task's business logic from its dependency declaration.

03

Unified Bridge

An external dataset-to-task mapping can be the bridge between task-centric and dataset-centric architectures.

04

In-House Platform

Leverage your in-house analytics platform if you already have one that users are using for data exploration and prototyping.

Careers at Stripe

We're hiring an Engineering Manager

If you're interested in career opportunities at Stripe, our Core Data Engineering & Infra team is looking for an EM to help shape the future of Data & AI.

Scan the QR code to apply, or visit:

<https://share.google/JK4daFlgSliaxfVPn>

stripe



Archana Ramachandran

Head of Core Data Engineering & Infra @
Stripe | Data & AI

